Lab 3 Report: QtPackMan MVC, Coding Standards, and Doxygen Documentation

Intelligent Agents and Simulation – Lab 3

1. Introduction

This report documents the work completed for Lab 3. The lab focuses on the Qt version of the Pac-Man project, with emphasis on the Model-View-Controller structure, coding standards, memory management, and Doxygen documentation support.

The original QtPackMan project already contains a graphical board implemented with Qt widgets, a model that stores the board state, and a controller that coordinates the game flow. The work completed in this lab improves documentation and maintainability rather than changing the basic gameplay rules.

2. Project Structure

The main source files are located in QtPackMan.d. The most relevant classes are listed below.

File / Class	Purpose
BoardObject.h	Base class for board objects such as Cookie, Player, and Wall.
Cookie.h / Cookie.cpp	Represents collectible cookies and their prize value.
Player.h / Player.cpp	Represents the player object on the board.
Wall.h / Wall.cpp	Represents blocking wall objects.
ListBoardObjects.h / .cpp	Stores BoardObject pointers in each board cell.
Model.h / Model.cpp	Stores and updates the game state.
BoardCells.h / .cpp	Represents the visible board layout using Qt cells.
GameCell.h / .cpp	Represents an individual visible cell in the Qt board.
Controller.h / .cpp	Coordinates user input, timer events, model updates, and board display.
main.cpp	Starts the Qt application and creates the controller.

3. Coding Standards and Memory Management Improvements

The practical material highlights the importance of improving code quality and avoiding memory-management errors. The original project uses dynamic allocation through new, so explicit deletion is required when objects are no longer needed.

The following improvements were made:

- A virtual destructor was added to BoardObject so that derived objects can be deleted safely through BoardObject pointers.
- A destructor was added to ListBoardObjects to delete the BoardObject pointers stored in its internal vector.
- A destructor was added to Model to delete the dynamically allocated ListBoardObjects objects stored in mySpace.

- The Controller destructor was extended to delete gameTimer, myModel, and myBoard, then set those pointers to NULL.

These changes reduce memory leaks and improve the safety of the object-oriented design.

3.1 BoardObject Destructor

```
virtual ~BoardObject() {};
```

Because BoardObject is used polymorphically, the destructor should be virtual. This allows the correct destructor chain to run when a derived object is deleted through a BoardObject pointer.

3.2 ListBoardObjects Destructor

```
ListBoardObjects::~ListBoardObjects()
{
   std::vector<p_BoardObject>::iterator it = list.begin();
   while (it != list.end())
   {
      delete (*it);
      *it = NULL;
      it++;
   }
   list.clear();
}
```

This destructor releases the BoardObject pointers stored in the list and then clears the vector.

3.3 Model and Controller Destructors

The Model destructor releases every ListBoardObjects cell stored in the two-dimensional mySpace array. The Controller destructor releases the timer, model, and board view. These changes match the ownership relationships shown in the UML diagram.

4. Doxygen Documentation

A Doxygen configuration file named Doxyfile was added to QtPackMan.d. This allows automatic HTML documentation to be generated from source files and comments.

The Makefile was also extended with a documentation target:

```
doc:
    doxygen Doxyfile
```

If Doxygen is installed, documentation can be generated from the QtPackMan.d folder using:

```
mingw32-make doc
```

or, depending on the local environment:

```
make doc
```

If Doxygen is not installed on the computer, the Doxyfile and Makefile target still document how project documentation can be generated in a configured environment.

5. UML Class Diagram

The UML class diagram shows the structure of the QtPackMan project. Cookie, Player, and Wall inherit from BoardObject. Model owns the board-state data, ListBoardObjects stores BoardObject pointers, and Controller coordinates the Model and BoardCells view.

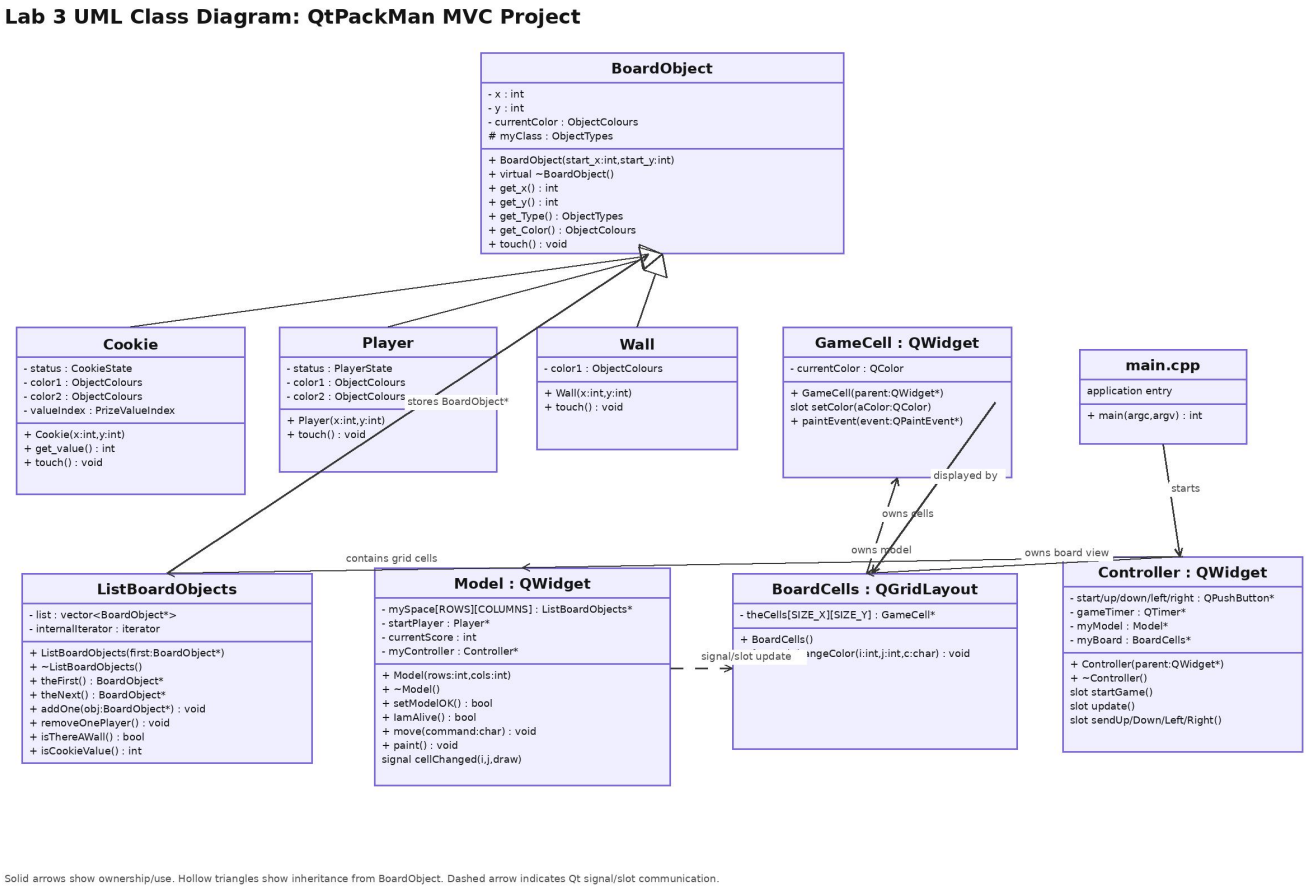

Figure 1. UML class diagram for the QtPackMan project.

6. MVC / Interaction Diagram

The project can be interpreted using the Model-View-Controller pattern. The Controller receives Qt button and timer events, the Model stores and updates game state, and the View displays the board using BoardCells and GameCell objects.

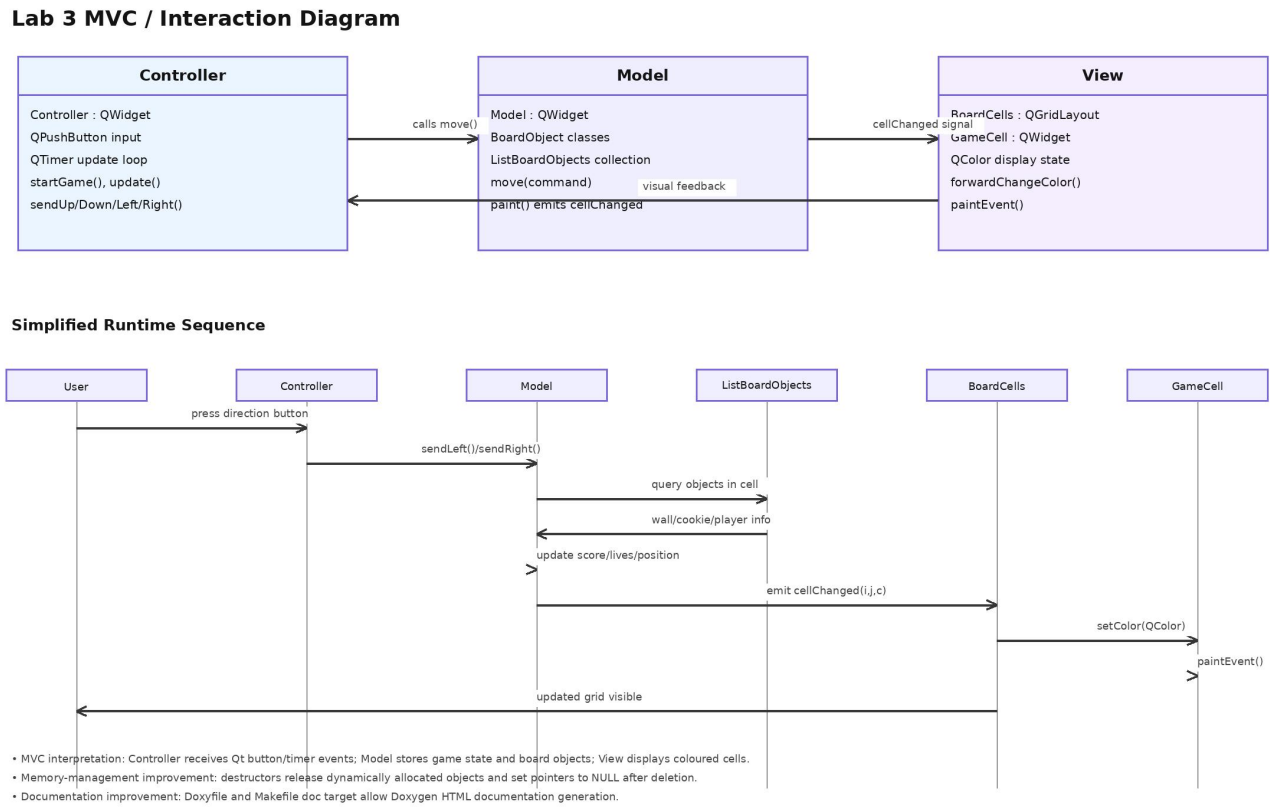

Figure 2. MVC and runtime interaction diagram.

7.Runtime Testing and Execution Evidence

7.1 Qt Build Environment

The QtPackMan application was built and executed using Qt Creator on Windows. The project was opened through the **QtPackMan.d.pro** file. This file is the Qt project configuration file used by the qmake build system to generate the required Makefile and compile the application.

To make the legacy Qt project compatible with the current Qt Creator environment, the **.pro** file was updated as follows:

TEMPLATE = app

TARGET = QtPackMan

QT += widgets

DEPENDPATH += .

INCLUDEPATH += .

The line **QT += widgets** was added because the application uses Qt Widget-based GUI components, including **QApplication,** layout widgets, buttons, and graphical board cells. The project was configured using the **Desktop Qt 6.11.1 MinGW 64-bit** kit.

7.2 Successful Application Launch

After the project was configured, it was built and executed successfully in Qt Creator. The first runtime screenshot shows the QtPackMan application window after launch. The window displays the **START** button and the main graphical area, confirming that the Qt GUI can be created and displayed correctly.

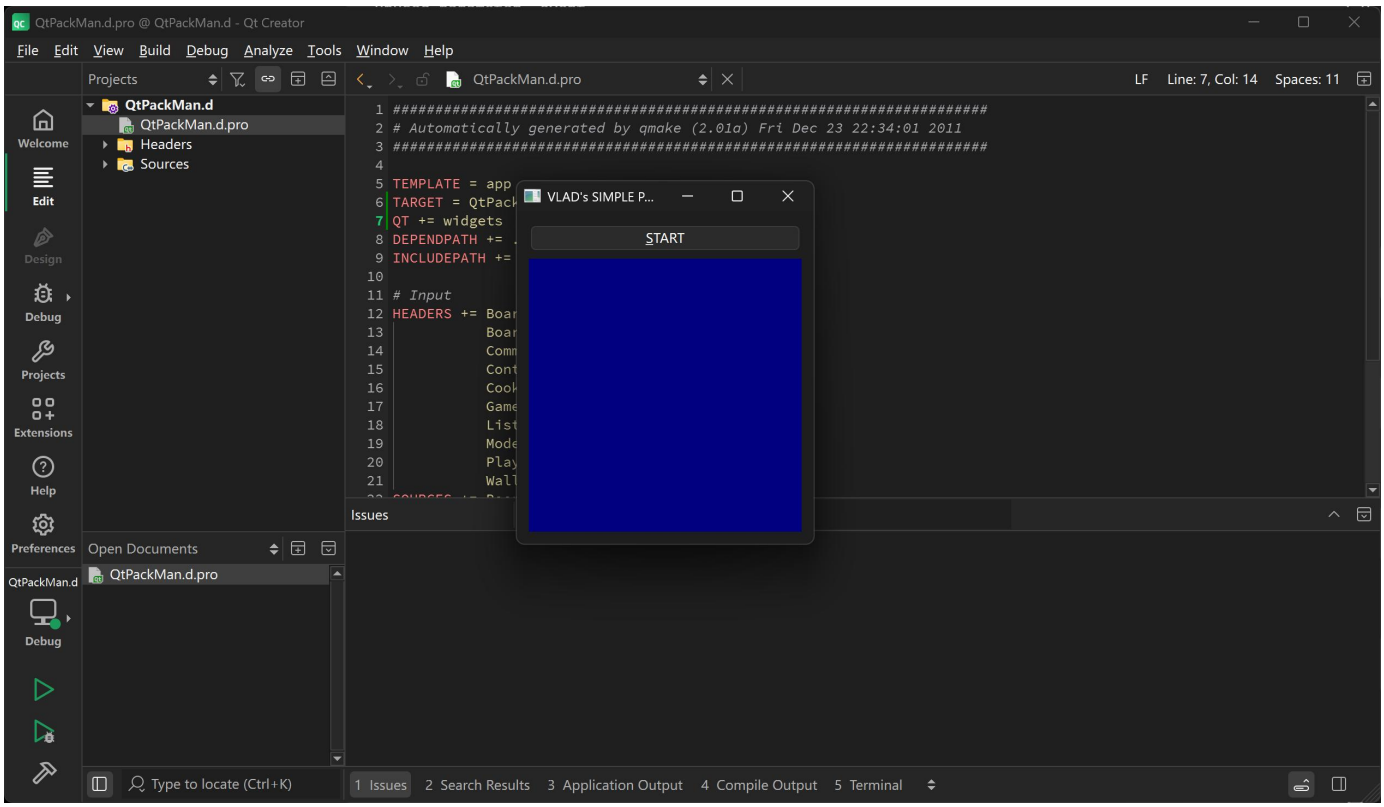

Figure 2. QtPackMan_start_screen.

7.3 GUI Runtime Interaction

After pressing the **START** button, the game interface displayed the board and movement controls. The GUI includes directional control buttons: **UP, DOWN, LEFT,** and **RIGHT.**

The board is rendered using coloured cells. The blue area represents the main board background, the black vertical blocks represent wall structures, and the coloured object cells represent interactive board objects. This demonstrates that the graphical board rendering is working during runtime.

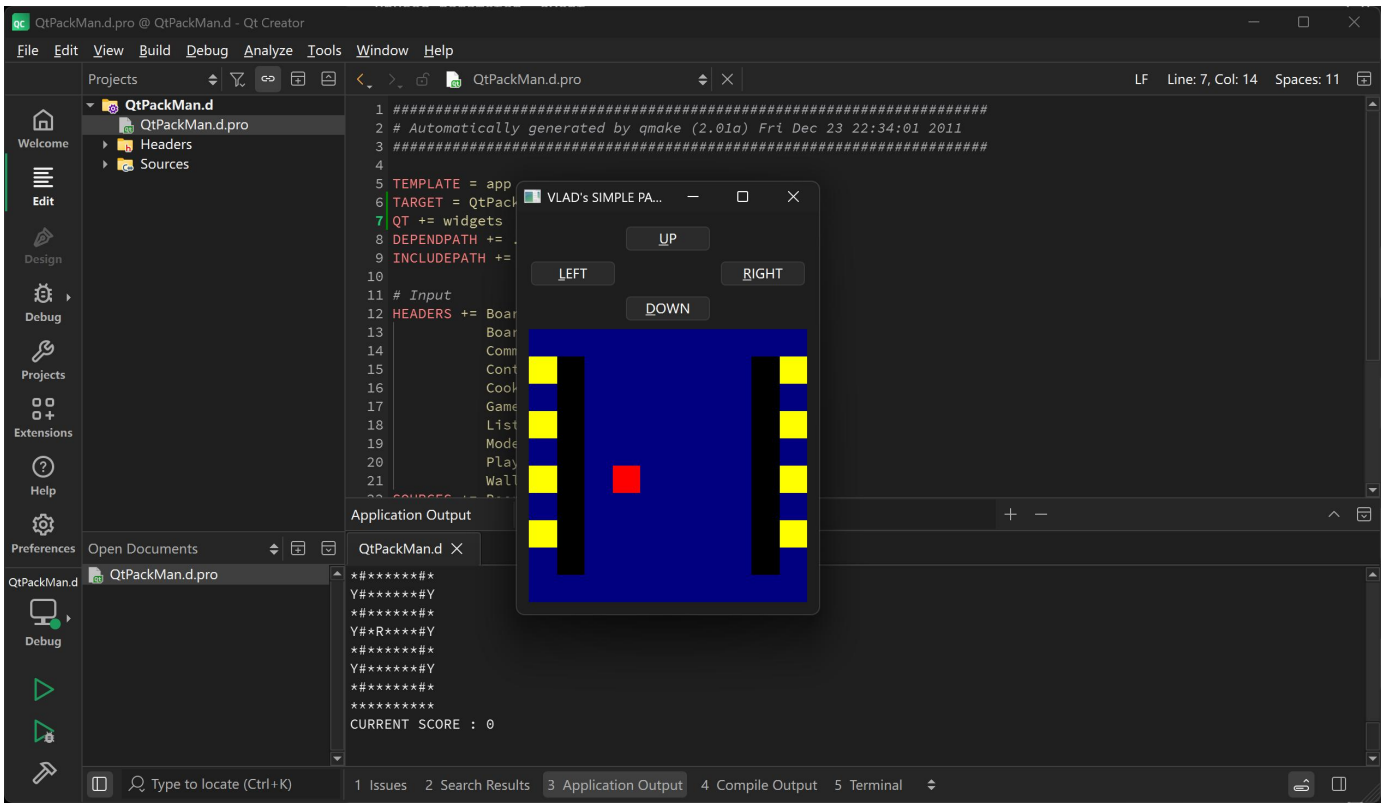

Figure 2. QtPackMan_runtime_score_0.

7.4 Score Update Evidence

The third runtime screenshot provides further evidence that the application is not only opening as a static window, but also executing its internal game logic. After interacting with the game, the Application Output panel shows that the current score increased from **0** to **40**.

This confirms that the main runtime loop, GUI update cycle, and game-state logic are functioning together. The runtime behaviour can be interpreted through the MVC structure:

- The Controller receives user input through the direction buttons.

- The Model updates the board state and score.

- The View refreshes the graphical board display.

- The Application Output panel prints the current game state and score.

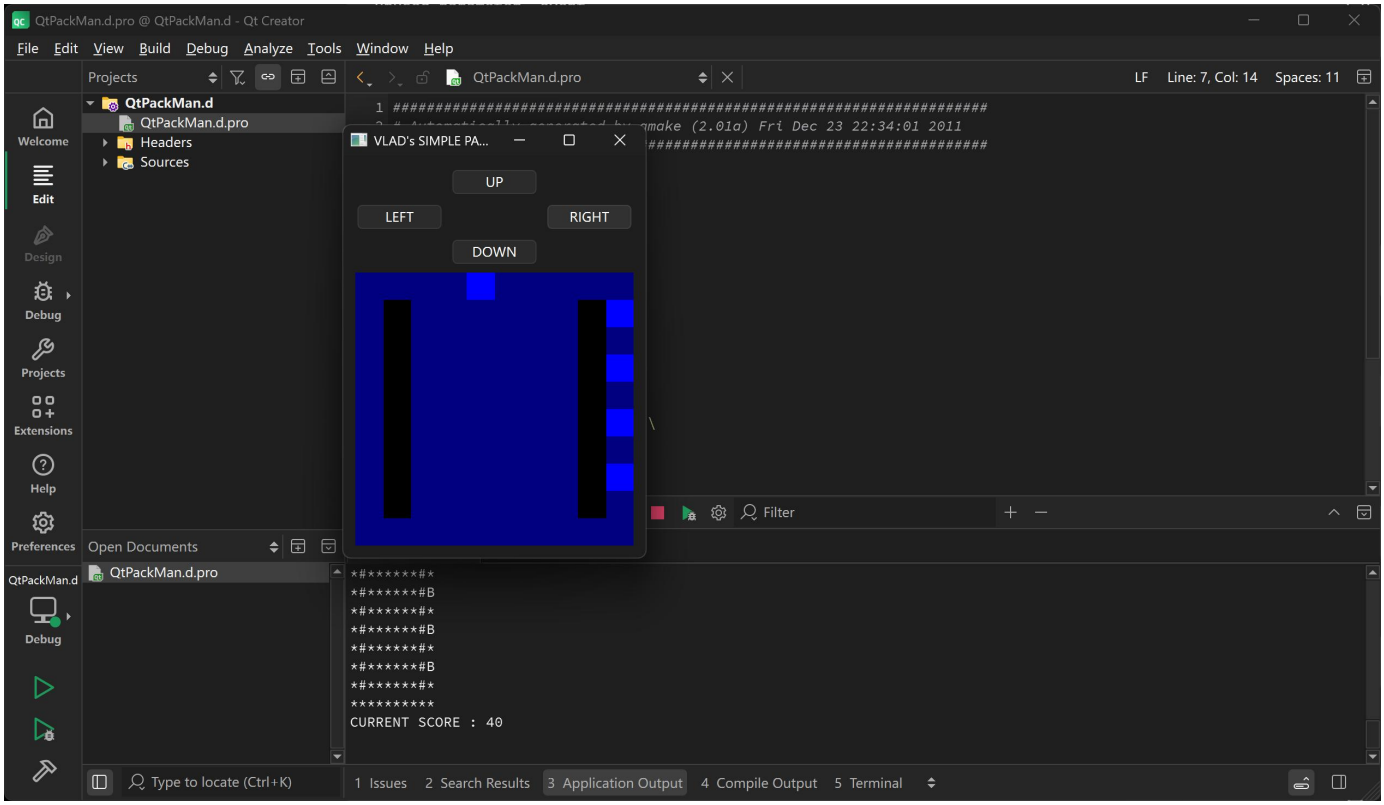

Figure 2. QtPackMan_runtime_score_40.

7.5 Runtime Testing Summary

The runtime test confirms that the QtPackMan GUI prototype can be built and executed successfully in Qt Creator. The screenshots provide evidence of application launch, GUI display, board rendering, movement controls, and score update.

Although the application is a simple educational PacMan prototype rather than a complete commercial game, the successful execution demonstrates the main Lab 3 objectives: using Qt for GUI programming, compiling a Qt project through a **.pro** file, observing runtime behaviour, and connecting the implementation to the MVC design.

8. Summary of Modified / Added Files

- BoardObject.h: added a virtual destructor.
- ListBoardObjects.h: added destructor declaration.
- ListBoardObjects.cpp: added destructor implementation to release stored BoardObject pointers.
- Model.h: added destructor declaration.
- Model.cpp: added destructor implementation to release allocated cell collections.
- Controller.cpp: updated destructor to release owned objects.
- Doxyfile: added Doxygen configuration.
- Makefile: added a doc target for Doxygen generation.
- QtPackMan.d.pro: updated TARGET and added QT += widgets for Qt Creator compatibility.
- screenshots/: added runtime evidence screenshots showing successful QtPackMan execution.

9. Conclusion

Lab 3 improved the QtPackMan project by analysing its MVC structure, documenting its class relationships, and applying basic coding-standard improvements. The memory-management changes address dynamically allocated objects, and the Doxygen configuration provides a path for automatic documentation generation. The UML and MVC diagrams clarify how the project is organised and how Qt events flow through the controller, model, and view components.